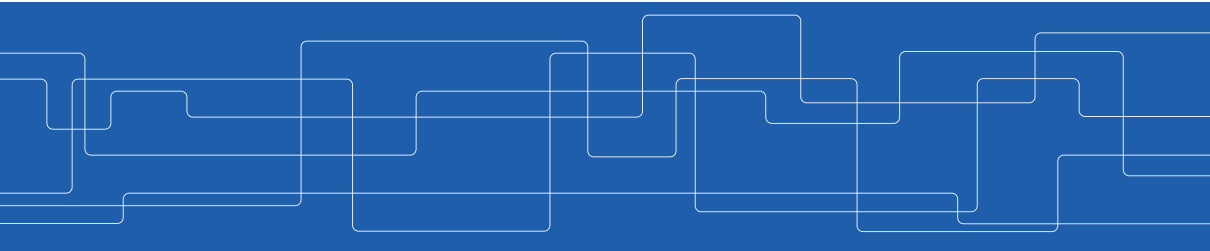


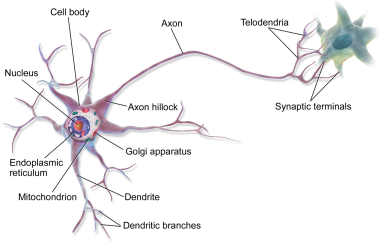
Deep Feedforwards Networks

Amir H. Payberah
payberah@kth.se
2026-11-04



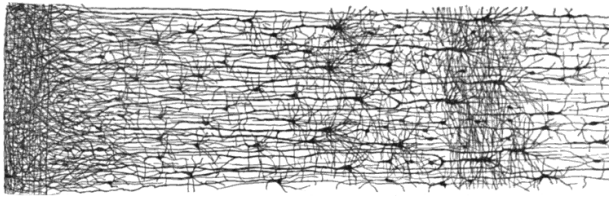
Biological Neurons (1/2)

- Brain architecture has inspired artificial neural networks.
- When a neuron receives a sufficient number of signals within a few milliseconds, it fires its own signals.



Biological Neurons (2/2)

- Biological neurons are organized in a vast network of billions of neurons.
- Each neuron typically is connected to thousands of other neurons.



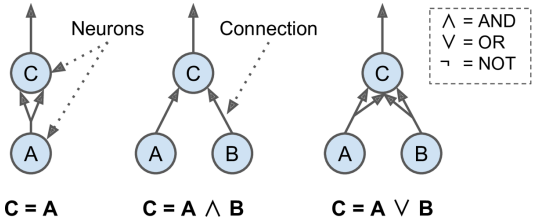


A Simple Artificial Neural Network

- One or more binary inputs and one binary output
- Activates its output when more than a certain number of its inputs are active.

A Simple Artificial Neural Network

- One or more binary inputs and one binary output
- Activates its output when more than a certain number of its inputs are active.



[A. Geron, O'Reilly Media, 2017]



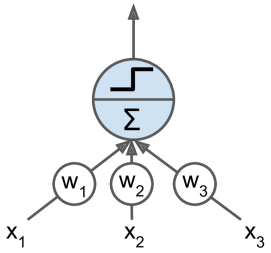
The Linear Threshold Unit (LTU)

- Inputs of a LTU are numbers (not binary).

The Linear Threshold Unit (LTU)

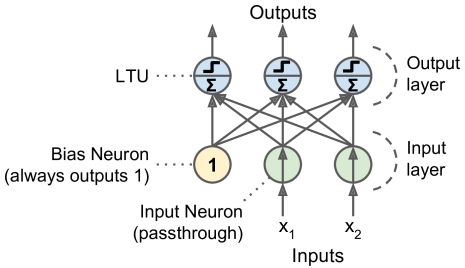
- Inputs of a LTU are numbers (not binary).
- Each input connection is associated with a weight.
- Computes a weighted sum of its inputs and applies a step function to that sum.

- $z = w_1 x_1 + w_2 x_2 + \cdots + w_n x_n = \mathbf{w}^\intercal \mathbf{x}$
- $\hat{y} = \text{step}(z) = \text{step}(\mathbf{w}^\intercal \mathbf{x})$



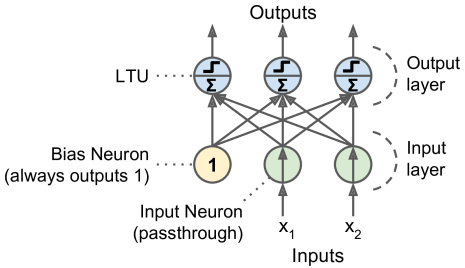
The Perceptron

- The perceptron is a single layer of LTUs.



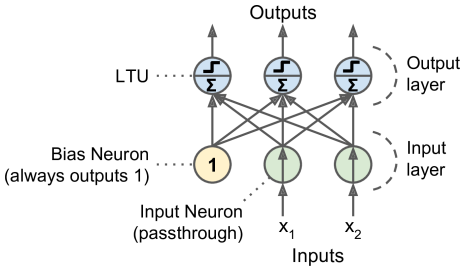
The Perceptron

- The perceptron is a single layer of LTUs.
- The input neurons output whatever input they are fed.



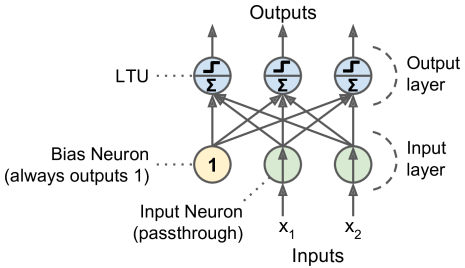
The Perceptron

- The perceptron is a single layer of LTUs.
- The input neurons output whatever input they are fed.
- A bias neuron, which just outputs 1 all the time.



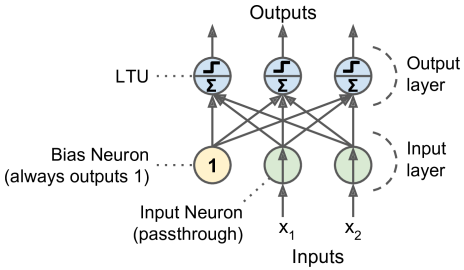
The Perceptron

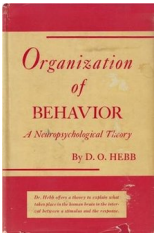
- The perceptron is a single layer of LTUs.
- The input neurons output whatever input they are fed.
- A bias neuron, which just outputs 1 all the time.
- If we use logistic function (sigmoid) instead of a step function, it computes a continuous output.



How is a Perceptron Trained? (1/2)

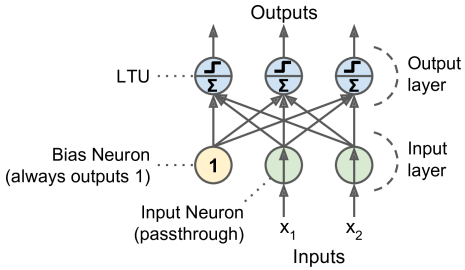
- The Perceptron training algorithm is inspired by Hebb's rule.

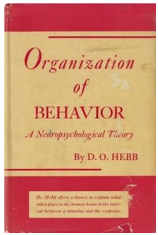




How is a Perceptron Trained? (1/2)

- The Perceptron training algorithm is inspired by Hebb's rule.
- When a biological neuron often triggers another neuron, the connection between these two neurons grows stronger.







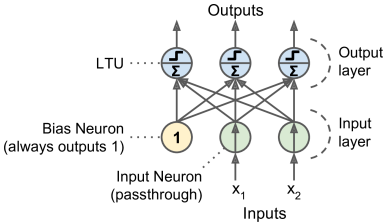
How is a Perceptron Trained? (2/2)

- Feed one training instance $\mathbf{x}$ to each neuron j at a time and make its prediction $\hat{y}$.
- Update the connection weights.

How is a Perceptron Trained? (2/2)

- Feed one training instance $\mathbf{x}$ to each neuron j at a time and make its prediction $\hat{y}$.
- Update the connection weights.

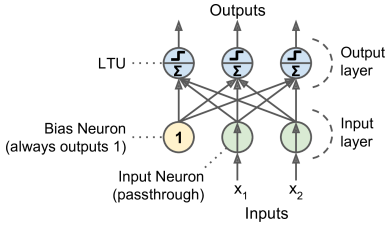
$$\hat{y}_j = \sigma(\mathbf{w}_j^T \mathbf{x} + b)$$
$$J(\mathbf{w}_j) = \texttt{cross_entropy}(y_j, \hat{y}_j)$$
$$w_{i,j}^{(\text{next})} = w_{i,j} - \eta \frac{\partial J(\mathbf{w}_j)}{w_i}$$



How is a Perceptron Trained? (2/2)

- Feed one training instance $\mathbf{x}$ to each neuron $\mathtt{j}$ at a time and make its prediction $\hat{\mathtt{y}}$.
- Update the connection weights.

$$\hat{\mathtt{y}}_{\mathtt{j}} = \sigma(\mathbf{w}_{\mathtt{j}}^{\mathsf{T}}\mathbf{x} + \mathtt{b})$$
$$\mathtt{J}(\mathbf{w}_{\mathtt{j}}) = \mathtt{cross_entropy}(\mathtt{y}_{\mathtt{j}}, \hat{\mathtt{y}}_{\mathtt{j}})$$
$$\mathtt{w}_{\mathtt{i},\mathtt{j}}^{(\mathtt{next})} = \mathtt{w}_{\mathtt{i},\mathtt{j}} - \eta \frac{\partial \mathtt{J}(\mathbf{w}_{\mathtt{j}})}{\mathtt{w}_{\mathtt{i}}}$$

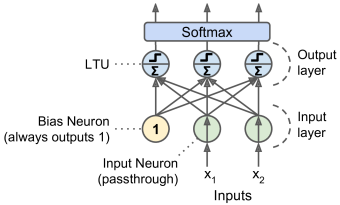


- $\mathtt{w}_{\mathtt{i},\mathtt{j}}$: the weight between neurons $\mathtt{i}$ and $\mathtt{j}$.
- $\mathtt{x}_{\mathtt{i}}$: the $\mathtt{i}$th input value of current instance.
- $\hat{\mathtt{y}}_{\mathtt{j}}$: the $\mathtt{j}$th predicted output value.
- $\mathtt{y}_{\mathtt{j}}$: the $\mathtt{j}$th true output value.
- η: the learning rate.

Perceptron in TensorFlow



Perceptron in TensorFlow

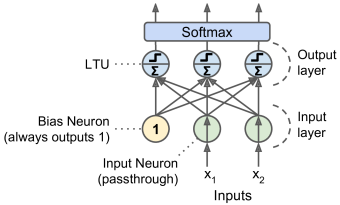



```
n_neurons = 3
n_features = 2

X_train = np.random.rand(100, n_features)
y_train = np.random.randint(0, n_neurons, size=(100,))

model = keras.models.Sequential()
model.add(keras.layers.Dense(n_neurons, input_shape=(n_features,), activation="softmax"))
```

Perceptron in TensorFlow




```
n_neurons = 3
n_features = 2

X_train = np.random.rand(100, n_features)
y_train = np.random.randint(0, n_neurons, size=(100,))

model = keras.models.Sequential()
model.add(keras.layers.Dense(n_neurons, input_shape=(n_features,), activation="softmax"))
```

```
model.compile(loss="sparse_categorical_crossentropy", optimizer="sgd", metrics=["accuracy"])
model.fit(X_train, y_train, epochs=30)
```

Multi-Layer Perceptron (MLP)



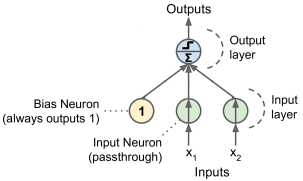
Perceptron Weakness (1/2)

- Incapable of solving some trivial problems, e.g., XOR classification problem. Why?

Perceptron Weakness (1/2)

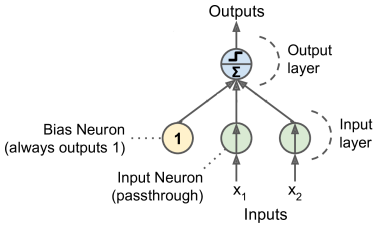
- Incapable of solving some trivial problems, e.g., XOR classification problem. Why?

A	B	A **XOR** B
0	0	0
0	1	1
1	0	1
1	1	0



$$\mathbf{X} = \begin{bmatrix} 0 & 0 \\ 0 & 1 \\ 1 & 0 \\ 1 & 1 \end{bmatrix} \qquad \mathbf{y} = \begin{bmatrix} 0 \\ 1 \\ 1 \\ 0 \end{bmatrix}$$

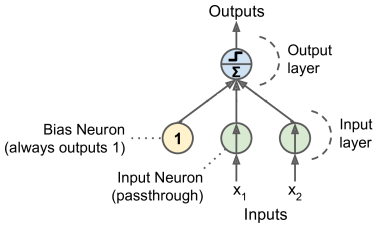
Perceptron Weakness (2/2)



$$\mathbf{X} = \begin{bmatrix} 0 & 0 \\ 0 & 1 \\ 1 & 0 \\ 1 & 1 \end{bmatrix} \qquad \mathbf{y} = \begin{bmatrix} 0 \\ 1 \\ 1 \\ 0 \end{bmatrix} \qquad \hat{y} = \mathtt{step}(z), z = w_1 x_1 + w_2 x_2 + b$$

$$J(\mathbf{w}) = \frac{1}{4} \sum_{x \in \mathbf{X}} (\hat{y}(x) - y(x))^2$$

Perceptron Weakness (2/2)

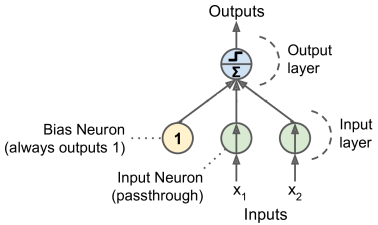


$$\mathbf{X} = \begin{bmatrix} 0 & 0 \\ 0 & 1 \\ 1 & 0 \\ 1 & 1 \end{bmatrix} \qquad \mathbf{y} = \begin{bmatrix} 0 \\ 1 \\ 1 \\ 0 \end{bmatrix} \qquad \hat{y} = \texttt{step}(z), z = w_1 x_1 + w_2 x_2 + b$$

$$J(\mathbf{w}) = \frac{1}{4} \sum_{x \in \mathbf{X}} (\hat{y}(x) - y(x))^2$$

- If we minimize $J(\mathbf{w})$, we obtain $w_1 = 0$, $w_2 = 0$, and $b = \frac{1}{2}$.

Perceptron Weakness (2/2)



$$\mathbf{X} = \begin{bmatrix} 0 & 0 \\ 0 & 1 \\ 1 & 0 \\ 1 & 1 \end{bmatrix} \qquad \mathbf{y} = \begin{bmatrix} 0 \\ 1 \\ 1 \\ 0 \end{bmatrix} \qquad \hat{y} = \mathtt{step}(z), z = w_1 x_1 + w_2 x_2 + b$$

$$J(\mathbf{w}) = \frac{1}{4} \sum_{x \in \mathbf{X}} (\hat{y}(x) - y(x))^2$$

- If we minimize $J(\mathbf{w})$, we obtain $w_1 = 0$, $w_2 = 0$, and $b = \frac{1}{2}$.
- But, the model outputs 0.5 everywhere.

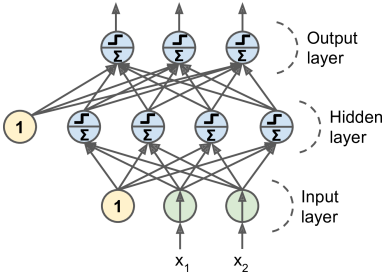


Multi-Layer Perceptron (MLP)

- The limitations of Perceptrons can be eliminated by stacking multiple Perceptrons.
- The resulting network is called a Multi-Layer Perceptron (MLP) or deep feedforward neural network.

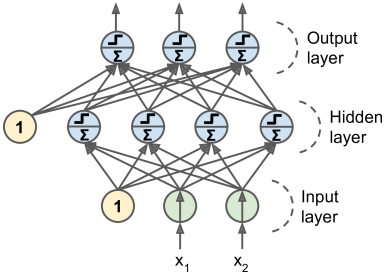
Feedforward Neural Network Architecture

- A feedforward neural network is composed of:
  - One input layer
  - One or more hidden layers
  - One final output layer

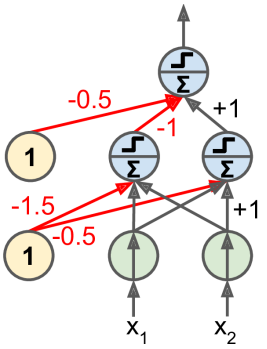


Feedforward Neural Network Architecture

- A feedforward neural network is composed of:
  - One input layer
  - One or more hidden layers
  - One final output layer
- Every layer except the output layer includes a bias neuron and is fully connected to the next layer.

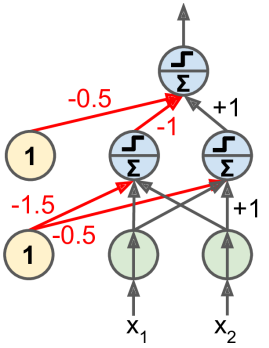


XOR with Feedforward Neural Network (1/3)



$$\mathbf{X} = \begin{bmatrix} 0 & 0 \\ 0 & 1 \\ 1 & 0 \\ 1 & 1 \end{bmatrix} \qquad \mathbf{y} = \begin{bmatrix} 0 \\ 1 \\ 1 \\ 0 \end{bmatrix} \qquad \mathbf{W}_x = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \qquad \mathbf{b}_x = \begin{bmatrix} -1.5 \\ -0.5 \end{bmatrix}$$

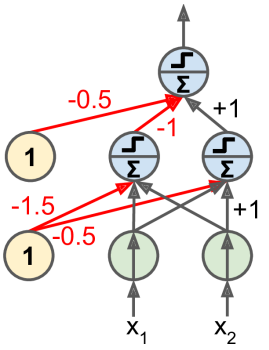
XOR with Feedforward Neural Network (2/3)



$$\mathbf{out}_h = \mathbf{X}\mathbf{W}_x^T + \mathbf{b}_x = \begin{bmatrix} -1.5 & -0.5 \\ -0.5 & 0.5 \\ -0.5 & 0.5 \\ 0.5 & 1.5 \end{bmatrix} \qquad \mathbf{h} = \mathtt{step}(\mathbf{out}_h) = \begin{bmatrix} 0 & 0 \\ 0 & 1 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}$$

$$\mathbf{w}_h = \begin{bmatrix} -1 \\ 1 \end{bmatrix} \qquad b_h = -0.5$$

XOR with Feedforward Neural Network (3/3)

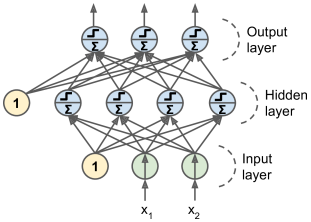


$$\mathbf{out} = \mathbf{w}_h^T \mathbf{h} + \mathrm{b}_h = \begin{bmatrix} -0.5 \\ 0.5 \\ 0.5 \\ -0.5 \end{bmatrix} \qquad \mathtt{step}(\mathbf{out}) = \begin{bmatrix} 0 \\ 1 \\ 1 \\ 0 \end{bmatrix}$$

How to Learn Model Parameters **W**?

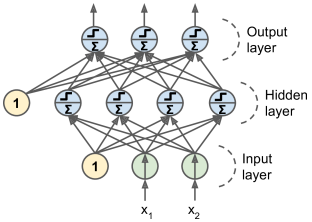
Training Feedforward Neural Networks

- How to train a feedforward neural network?



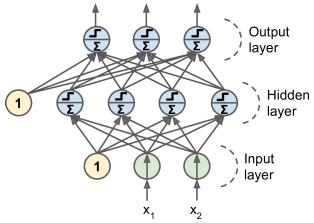
Training Feedforward Neural Networks

- How to train a feedforward neural network?
- For each training instance $\mathbf{x}^{(i)}$ the algorithm does the following steps:



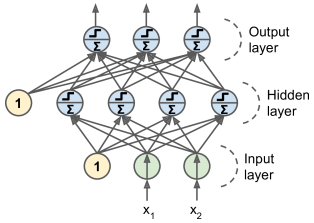
Training Feedforward Neural Networks

- How to train a feedforward neural network?
- For each training instance $\mathbf{x}^{(i)}$ the algorithm does the following steps:
  1. Forward pass: make a prediction (compute $\hat{\mathbf{y}}^{(i)} = \mathbf{f}(\mathbf{x}^{(i)})$).



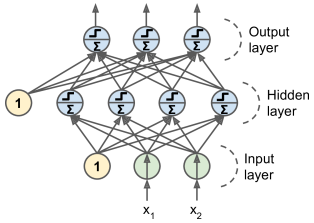
Training Feedforward Neural Networks

- How to train a feedforward neural network?
- For each training instance $\mathbf{x}^{(i)}$ the algorithm does the following steps:
  1. Forward pass: make a prediction (compute $\hat{\mathbf{y}}^{(i)} = \mathtt{f}(\mathbf{x}^{(i)})$).
  2. Measure the error (compute $\mathtt{cost}(\hat{\mathbf{y}}^{(i)}, \mathbf{y}^{(i)})$).



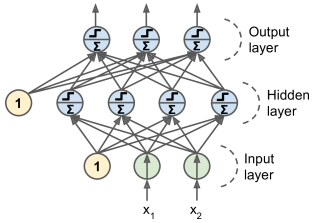
Training Feedforward Neural Networks

- How to train a feedforward neural network?
- For each training instance $\mathbf{x}^{(i)}$ the algorithm does the following steps:
  1. Forward pass: make a prediction (compute $\hat{\mathbf{y}}^{(i)} = \mathtt{f}(\mathbf{x}^{(i)})$).
  2. Measure the error (compute $\mathtt{cost}(\hat{\mathbf{y}}^{(i)}, \mathbf{y}^{(i)})$).
  3. Backward pass: go through each layer in reverse to measure the error contribution from each connection.



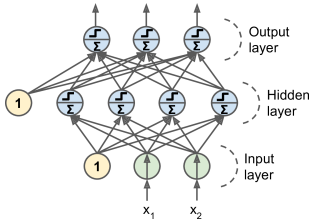
Training Feedforward Neural Networks

- How to train a feedforward neural network?
- For each training instance $\mathbf{x}^{(i)}$ the algorithm does the following steps:
  1. Forward pass: make a prediction (compute $\hat{\mathbf{y}}^{(i)} = \mathtt{f}(\mathbf{x}^{(i)})$).
  2. Measure the error (compute $\mathtt{cost}(\hat{\mathbf{y}}^{(i)}, \mathbf{y}^{(i)})$).
  3. Backward pass: go through each layer in reverse to measure the error contribution from each connection.
  4. Tweak the connection weights to reduce the error (update $\mathbf{W}$ and $\mathbf{b}$).



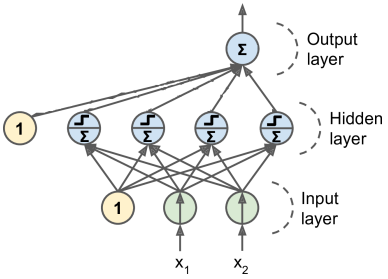
Training Feedforward Neural Networks

- How to train a feedforward neural network?
- For each training instance $\mathbf{x}^{(i)}$ the algorithm does the following steps:
  1. Forward pass: make a prediction (compute $\hat{\mathbf{y}}^{(i)} = \mathtt{f}(\mathbf{x}^{(i)})$).
  2. Measure the error (compute $\mathtt{cost}(\hat{\mathbf{y}}^{(i)}, \mathbf{y}^{(i)})$).
  3. Backward pass: go through each layer in reverse to measure the error contribution from each connection.
  4. Tweak the connection weights to reduce the error (update $\mathbf{W}$ and $\mathbf{b}$).
- It's called the backpropagation training algorithm



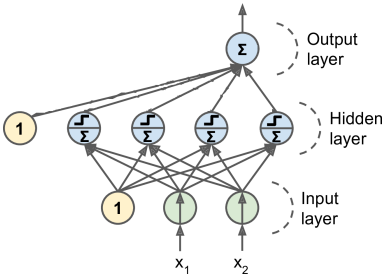
Output Unit (1/3)

- Linear units in neurons of the output layer.



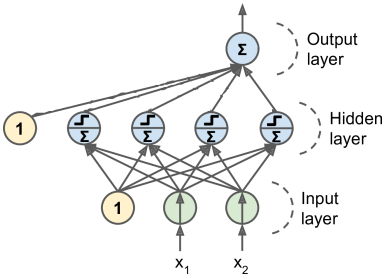
Output Unit (1/3)

- Linear units in neurons of the output layer.
- Output function: $\hat{y}_j = \mathbf{w}_j^T \mathbf{h} + b_j$.



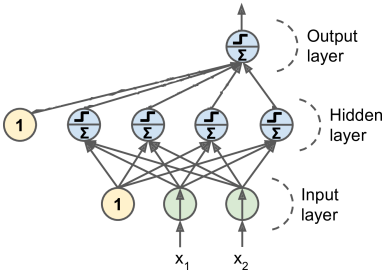
Output Unit (1/3)

- Linear units in neurons of the output layer.
- Output function: $\hat{y}_j = \mathbf{w}_j^T \mathbf{h} + b_j$.
- Cost function: minimizing the mean squared error.



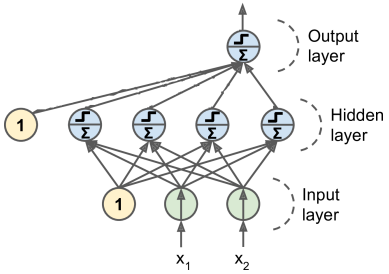
Output Unit (2/3)

- Sigmoid units in neurons of the output layer (binomial classification).



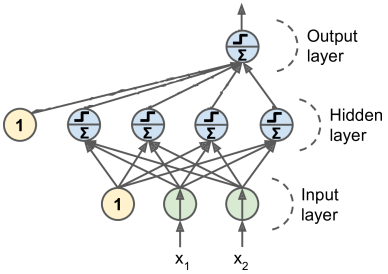
Output Unit (2/3)

- Sigmoid units in neurons of the output layer (binomial classification).
- Output function: $\hat{y}_j = \sigma(\mathbf{w}_j^T \mathbf{h} + b_j)$.



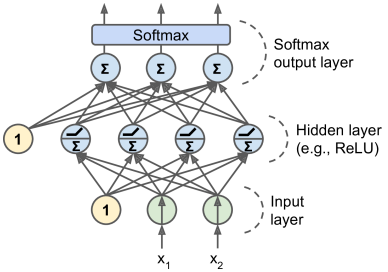
Output Unit (2/3)

- Sigmoid units in neurons of the output layer (binomial classification).
- Output function: $\hat{y}_j = \sigma(\mathbf{w}_j^T \mathbf{h} + b_j)$.
- Cost function: minimizing the cross-entropy, $\mathtt{cost}(y, \hat{y}) = -\sum_j y_j \mathtt{log}(\hat{y}_j)$



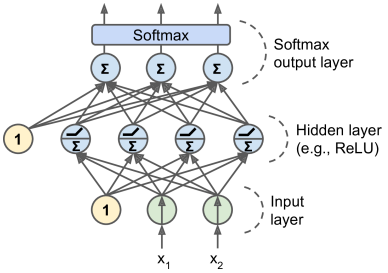
Output Unit (3/3)

- Softmax units in neurons of the output layer (multinomial classification).



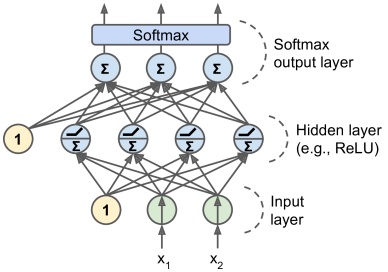
Output Unit (3/3)

- Softmax units in neurons of the output layer (multinomial classification).
- Output function: $\hat{y}_j = \texttt{softmax}(\mathbf{w}_j^T \mathbf{h} + b_j)$.



Output Unit (3/3)

- Softmax units in neurons of the output layer (multinomial classification).
- Output function: $\hat{\mathbf{y}}_j = \mathtt{softmax}(\mathbf{w}_j^\intercal \mathbf{h} + \mathtt{b}_j)$.
- Cost function: minimizing the cross-entropy.



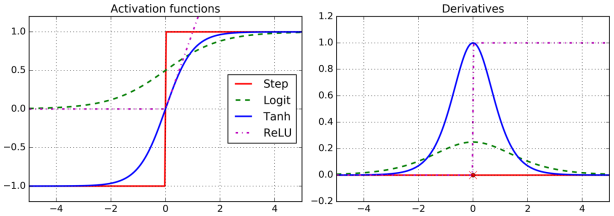


Hidden Units

- In order for the backpropagation algorithm to work properly, we need to replace the step function with other activation functions. Why?

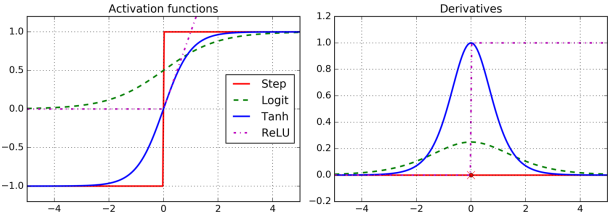
Hidden Units

- In order for the backpropagation algorithm to work properly, we need to replace the step function with other activation functions. Why?
- Alternative activation functions:



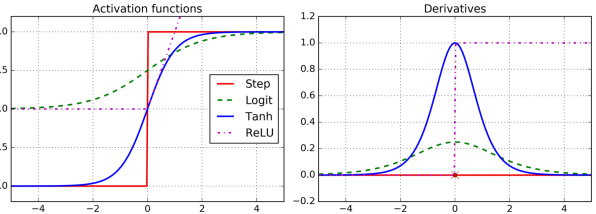
Hidden Units

- In order for the backpropagation algorithm to work properly, we need to replace the step function with other activation functions. Why?
- Alternative activation functions:
  1. Logistic function (sigmoid): $\sigma(z) = \frac{1}{1+e^{-z}}$



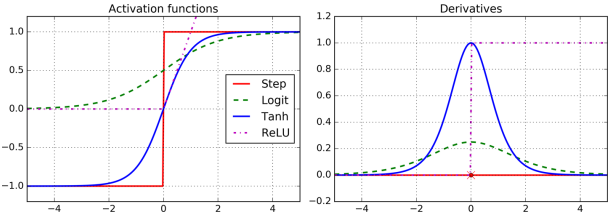
Hidden Units

- In order for the backpropagation algorithm to work properly, we need to replace the step function with other activation functions. Why?
- Alternative activation functions:
  1. Logistic function (sigmoid): $\sigma(z) = \frac{1}{1+e^{-z}}$
  2. Hyperbolic tangent function: $\tanh(z) = 2\sigma(2z) - 1$

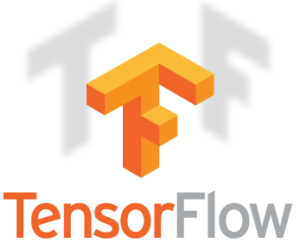


Hidden Units

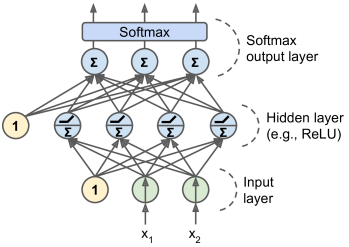
- In order for the backpropagation algorithm to work properly, we need to replace the step function with other activation functions. Why?
- Alternative activation functions:
  1. Logistic function (sigmoid): $\sigma(z) = \frac{1}{1+e^{-z}}$
  2. Hyperbolic tangent function: $\tanh(z) = 2\sigma(2z) - 1$
  3. Rectified linear units (ReLUs): $\mathrm{ReLU}(z) = \max(0, z)$



Feedforward Network in TensorFlow



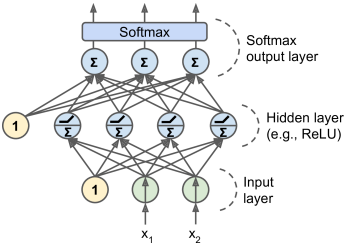
Feedforward Network in TensorFlow




```
n_output = 3
n_hidden = 4
n_features = 2

model = keras.models.Sequential()
model.add(keras.layers.Dense(n_hidden, input_shape=(n_features,), activation="relu"))
model.add(keras.layers.Dense(n_output, activation="softmax"))
```

Feedforward Network in TensorFlow



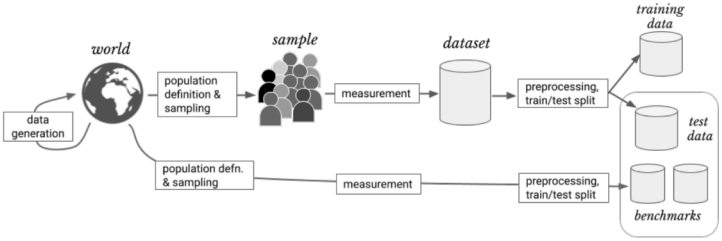

```
n_output = 3
n_hidden = 4
n_features = 2

model = keras.models.Sequential()
model.add(keras.layers.Dense(n_hidden, input_shape=(n_features,), activation="relu"))
model.add(keras.layers.Dense(n_output, activation="softmax"))
```

```
model.compile(loss="sparse_categorical_crossentropy", optimizer="sgd", metrics=["accuracy"])
model.fit(X_train, y_train, epochs=30)
```



ML Systems Life Cycle

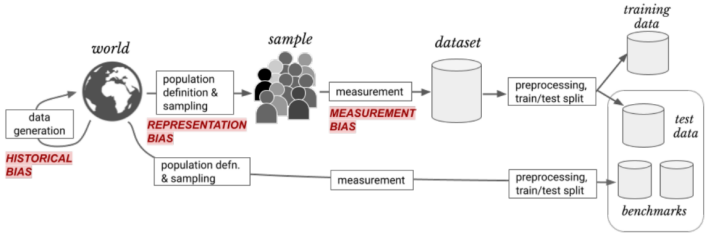


(a) Data Generation

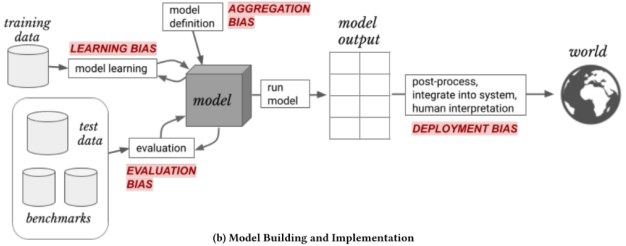
(b) Model Building and Implementation

H. Suresh et al., A Framework for Understanding Sources of Harm throughout the Machine Learning Life Cycle, 2021

Bias in ML Systems Life Cycle



(a) Data Generation



(b) Model Building and Implementation

H. Suresh et al., A Framework for Understanding Sources of Harm throughout the Machine Learning Life Cycle, 2021



Dive into Backpropagation Algorithm



Chain Rule of Calculus (1/2)

- Assume $\mathtt{x} \in \mathbb{R}$, and two functions $\mathtt{f}$ and $\mathtt{g}$, and also assume $\mathtt{y} = \mathtt{g}(\mathtt{x})$ and $\mathtt{z} = \mathtt{f}(\mathtt{y}) = \mathtt{f}(\mathtt{g}(\mathtt{x}))$.



Chain Rule of Calculus (1/2)

- Assume $\mathtt{x} \in \mathbb{R}$, and two functions $\mathtt{f}$ and $\mathtt{g}$, and also assume $\mathtt{y} = \mathtt{g}(\mathtt{x})$ and $\mathtt{z} = \mathtt{f}(\mathtt{y}) = \mathtt{f}(\mathtt{g}(\mathtt{x}))$.
- The chain rule of calculus is used to compute the derivatives of functions, e.g., $\mathtt{z}$, formed by composing other functions, e.g., $\mathtt{g}$.



Chain Rule of Calculus (1/2)

- Assume $x \in \mathbb{R}$, and two functions f and g, and also assume $y = g(x)$ and $z = f(y) = f(g(x))$.
- The chain rule of calculus is used to compute the derivatives of functions, e.g., z, formed by composing other functions, e.g., g.
- Then the chain rule states that $\frac{dz}{dx} = \frac{dz}{dy}\frac{dy}{dx}$



Chain Rule of Calculus (1/2)

- Assume $x \in \mathbb{R}$, and two functions f and g, and also assume $y = g(x)$ and $z = f(y) = f(g(x))$.
- The chain rule of calculus is used to compute the derivatives of functions, e.g., z, formed by composing other functions, e.g., g.
- Then the chain rule states that $\frac{dz}{dx} = \frac{dz}{dy}\frac{dy}{dx}$
- Example:

$$z = f(y) = 5y^4 \text{ and } y = g(x) = x^3 + 7$$

Chain Rule of Calculus (1/2)

- Assume $x \in \mathbb{R}$, and two functions f and g, and also assume $y = g(x)$ and $z = f(y) = f(g(x))$.
- The chain rule of calculus is used to compute the derivatives of functions, e.g., z, formed by composing other functions, e.g., g.
- Then the chain rule states that $\frac{dz}{dx} = \frac{dz}{dy}\frac{dy}{dx}$
- Example:

$$z = f(y) = 5y^4 \text{ and } y = g(x) = x^3 + 7$$

$$\frac{dz}{dx} = \frac{dz}{dy}\frac{dy}{dx}$$

Chain Rule of Calculus (1/2)

- Assume $x \in \mathbb{R}$, and two functions f and g, and also assume $y = g(x)$ and $z = f(y) = f(g(x))$.
- The chain rule of calculus is used to compute the derivatives of functions, e.g., z, formed by composing other functions, e.g., g.
- Then the chain rule states that $\frac{dz}{dx} = \frac{dz}{dy}\frac{dy}{dx}$
- Example:

$$z = f(y) = 5y^4 \text{ and } y = g(x) = x^3 + 7$$

$$\frac{dz}{dx} = \frac{dz}{dy}\frac{dy}{dx}$$

$$\frac{dz}{dy} = 20y^3 \text{ and } \frac{dy}{dx} = 3x^2$$

Chain Rule of Calculus (1/2)

- Assume $x \in \mathbb{R}$, and two functions f and g, and also assume $y = g(x)$ and $z = f(y) = f(g(x))$.
- The chain rule of calculus is used to compute the derivatives of functions, e.g., z, formed by composing other functions, e.g., g.
- Then the chain rule states that $\frac{dz}{dx} = \frac{dz}{dy}\frac{dy}{dx}$
- Example:

$$z = f(y) = 5y^4 \text{ and } y = g(x) = x^3 + 7$$

$$\frac{dz}{dx} = \frac{dz}{dy}\frac{dy}{dx}$$

$$\frac{dz}{dy} = 20y^3 \text{ and } \frac{dy}{dx} = 3x^2$$

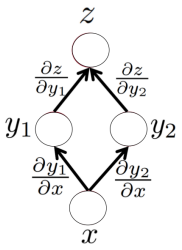
$$\frac{dz}{dx} = 20y^3 \times 3x^2 = 20(x^3 + 7) \times 3x^2$$

Chain Rule of Calculus (2/2)

- Two paths chain rule.

$$\mathtt{z} = \mathtt{f}(\mathtt{y}_1, \mathtt{y}_2) \text{ where } \mathtt{y}_1 = \mathtt{g}(\mathtt{x}) \text{ and } \mathtt{y}_2 = \mathtt{h}(\mathtt{x})$$

$$\frac{\partial \mathtt{z}}{\partial \mathtt{x}} = \frac{\partial \mathtt{z}}{\partial \mathtt{y}_1}\frac{\partial \mathtt{y}_1}{\partial \mathtt{x}} + \frac{\partial \mathtt{z}}{\partial \mathtt{y}_2}\frac{\partial \mathtt{y}_2}{\partial \mathtt{x}}$$



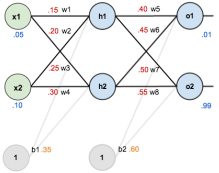


Backpropagation

- Backpropagation training algorithm for MLPs
- The algorithm repeats the following steps:
  1. Forward pass
  2. Backward pass

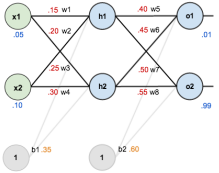
Backpropagation - Forward Pass

- Calculates outputs given input patterns.



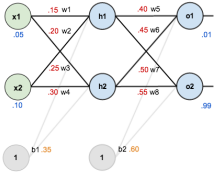
Backpropagation - Forward Pass

- Calculates outputs given input patterns.
- For each training instance



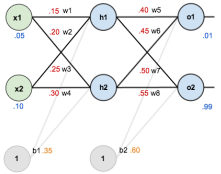
Backpropagation - Forward Pass

- Calculates outputs given input patterns.
- For each training instance
  - Feeds it to the network and computes the output of every neuron in each consecutive layer.



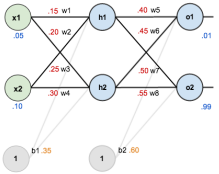
Backpropagation - Forward Pass

- Calculates outputs given input patterns.
- For each training instance
  - Feeds it to the network and computes the output of every neuron in each consecutive layer.
  - Measures the network's output error (i.e., the difference between the true and the predicted output of the network)



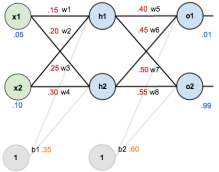
Backpropagation - Forward Pass

- Calculates outputs given input patterns.
- For each training instance
  - Feeds it to the network and computes the output of every neuron in each consecutive layer.
  - Measures the network's output error (i.e., the difference between the true and the predicted output of the network)
  - Computes how much each neuron in the last hidden layer contributed to each output neuron's error.



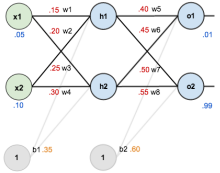
Backpropagation - Backward Pass

- Updates weights by calculating gradients.



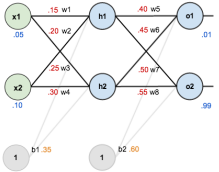
Backpropagation - Backward Pass

- Updates weights by calculating gradients.
- Measures how much of these error contributions came from each neuron in the previous hidden layer
  - Proceeds until the algorithm reaches the input layer.



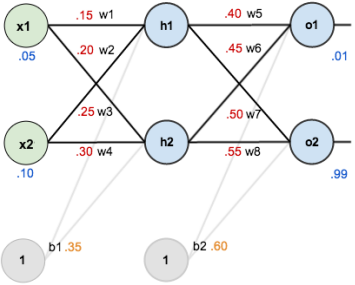
Backpropagation - Backward Pass

- Updates weights by calculating gradients.
- Measures how much of these error contributions came from each neuron in the previous hidden layer
  - Proceeds until the algorithm reaches the input layer.
- The last step is the gradient descent step on all the connection weights in the network, using the error gradients measured earlier.



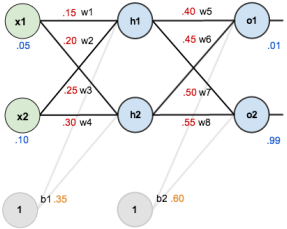
Backpropagation Example

- Two inputs, two hidden, and two output neurons.
- Bias in hidden and output neurons.
- Logistic activation in all the neurons.
- Squared error function as the cost function.



Backpropagation - Forward Pass (1/3)

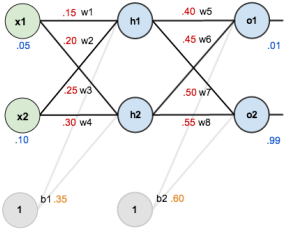
- Compute the output of the hidden layer



$$\texttt{net}_{\texttt{h1}} = \texttt{w}_1\texttt{x}_1 + \texttt{w}_2\texttt{x}_2 + \texttt{b}_1 = 0.15 \times 0.05 + 0.2 \times 0.1 + 0.35 = 0.3775$$

Backpropagation - Forward Pass (1/3)

- Compute the output of the hidden layer



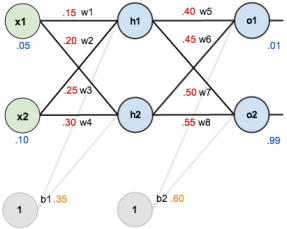
$$\texttt{net}_{\texttt{h1}} = \texttt{w}_1\texttt{x}_1 + \texttt{w}_2\texttt{x}_2 + \texttt{b}_1 = 0.15 \times 0.05 + 0.2 \times 0.1 + 0.35 = 0.3775$$

$$\texttt{out}_{\texttt{h1}} = \frac{1}{1 + \texttt{e}^{\texttt{net}_{\texttt{h1}}}} = \frac{1}{1 + \texttt{e}^{0.3775}} = 0.59327$$

$$\texttt{out}_{\texttt{h2}} = 0.59688$$

Backpropagation - Forward Pass (2/3)

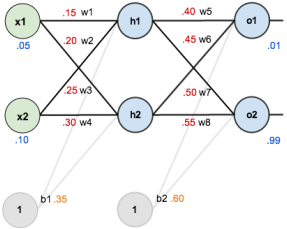
- Compute the output of the output layer



$$\texttt{net}_{o1} = \texttt{w}_5\texttt{out}_{h1} + \texttt{w}_6\texttt{out}_{h2} + \texttt{b}_2 = 0.4 \times 0.59327 + 0.45 \times 0.59688 + 0.6 = 1.1059$$

Backpropagation - Forward Pass (2/3)

- Compute the output of the output layer



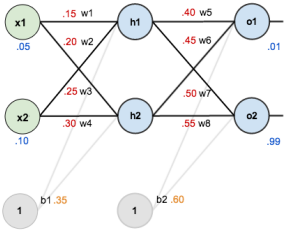
$$\texttt{net}_{o1} = \texttt{w}_5\texttt{out}_{h1} + \texttt{w}_6\texttt{out}_{h2} + \texttt{b}_2 = 0.4 \times 0.59327 + 0.45 \times 0.59688 + 0.6 = 1.1059$$

$$\texttt{out}_{o1} = \frac{1}{1 + e^{\texttt{net}_{o1}}} = \frac{1}{1 + e^{1.1059}} = 0.75136$$

$$\texttt{out}_{o2} = 0.77292$$

Backpropagation - Forward Pass (3/3)

- Calculate the error for each output



$$\mathrm{E_{o1}} = \frac{1}{2}(\mathrm{target_{o1}} - \mathrm{output_{o1}})^2 = \frac{1}{2}(0.01 - 0.75136)^2 = 0.27481$$

$$\mathrm{E_{o2}} = 0.02356$$

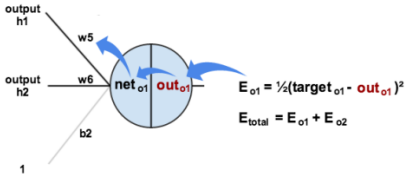
$$\mathrm{E_{total}} = \sum \frac{1}{2}(\mathrm{target} - \mathrm{output})^2 = \mathrm{E_{o1}} + \mathrm{E_{o2}} = 0.27481 + 0.02356 = 0.29837$$



[http://marimancusi.blogspot.com/2015/09/are-you-book-dragon.html]

Backpropagation - Backward Pass - Output Layer (1/6)

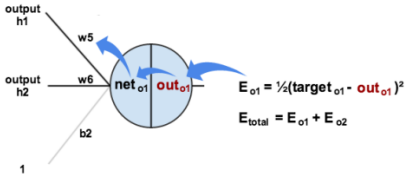
- Consider w_5
- We want to know how much a change in w_5 affects the total error ($\frac{\partial E_{total}}{\partial w_5}$)
- Applying the chain rule



$$\frac{\partial E_{total}}{\partial w_5} = \frac{\partial E_{total}}{\partial out_{o1}} \times \frac{\partial out_{o1}}{\partial net_{o1}} \times \frac{\partial net_{o1}}{\partial w_5}$$

Backpropagation - Backward Pass - Output Layer (2/6)

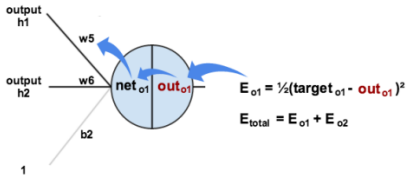
- First, how much does the total error change with respect to the output? ($\frac{\partial E_{total}}{\partial out_{o1}}$)



$$\frac{\partial E_{total}}{\partial w_5} = \frac{\partial E_{total}}{\partial out_{o1}} \times \frac{\partial out_{o1}}{\partial net_{o1}} \times \frac{\partial net_{o1}}{\partial w_5}$$

Backpropagation - Backward Pass - Output Layer (2/6)

- First, how much does the total error change with respect to the output? ($\frac{\partial E_{total}}{\partial out_{o1}}$)



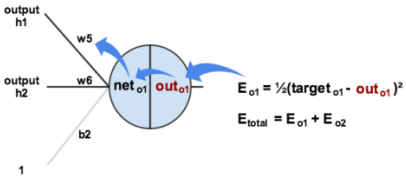
$$\frac{\partial E_{total}}{\partial w_5} = \frac{\partial E_{total}}{\partial out_{o1}} \times \frac{\partial out_{o1}}{\partial net_{o1}} \times \frac{\partial net_{o1}}{\partial w_5}$$

$$E_{total} = \frac{1}{2}(target_{o1} - out_{o1})^2 + \frac{1}{2}(target_{o2} - out_{o2})^2$$

$$\frac{\partial E_{total}}{\partial out_{o1}} = -2\frac{1}{2}(target_{o1} - out_{o1}) = -(0.01 - 0.75136) = 0.74136$$

Backpropagation - Backward Pass - Output Layer (3/6)

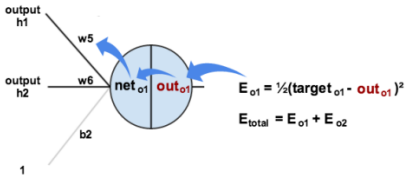
- Next, how much does the $\mathtt{out}_{o1}$ change with respect to its total input $\mathtt{net}_{o1}$? ($\frac{\partial \mathtt{out}_{o1}}{\partial \mathtt{net}_{o1}}$)



$$\frac{\partial \mathtt{E}_{\mathtt{total}}}{\partial \mathtt{w}_5} = \frac{\partial \mathtt{E}_{\mathtt{total}}}{\partial \mathtt{out}_{o1}} \times \frac{\partial \mathtt{out}_{o1}}{\partial \mathtt{net}_{o1}} \times \frac{\partial \mathtt{net}_{o1}}{\partial \mathtt{w}_5}$$

Backpropagation - Backward Pass - Output Layer (3/6)

- Next, how much does the out_{o1} change with respect to its total input net_{o1}? $(\frac{\partial \text{out}_{o1}}{\partial \text{net}_{o1}})$



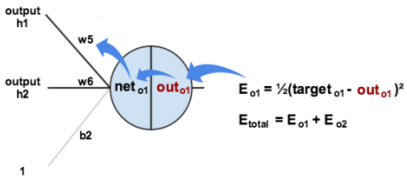
$$\frac{\partial \text{E}_{\text{total}}}{\partial \text{w}_5} = \frac{\partial \text{E}_{\text{total}}}{\partial \text{out}_{o1}} \times \frac{\partial \text{out}_{o1}}{\partial \text{net}_{o1}} \times \frac{\partial \text{net}_{o1}}{\partial \text{w}_5}$$

$$\text{out}_{o1} = \frac{1}{1 + \text{e}^{-\text{net}_{o1}}}$$

$$\frac{\partial \text{out}_{o1}}{\partial \text{net}_{o1}} = \text{out}_{o1}(1 - \text{out}_{o1}) = 0.75136(1 - 0.75136) = 0.18681$$

Backpropagation - Backward Pass - Output Layer (4/6)

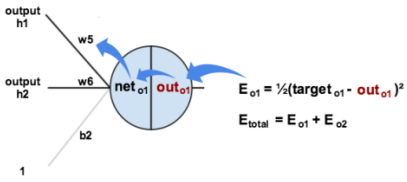
- Finally, how much does the total $\mathtt{net}_{o1}$ change with respect to $\mathtt{w}_5$? ($\frac{\partial \mathtt{net}_{o1}}{\partial \mathtt{w}_5}$)



$$\frac{\partial \mathtt{E}_{\mathtt{total}}}{\partial \mathtt{w}_5} = \frac{\partial \mathtt{E}_{\mathtt{total}}}{\partial \mathtt{out}_{o1}} \times \frac{\partial \mathtt{out}_{o1}}{\partial \mathtt{net}_{o1}} \times \frac{\partial \mathtt{net}_{o1}}{\partial \mathtt{w}_5}$$

Backpropagation - Backward Pass - Output Layer (4/6)

- Finally, how much does the total $\mathtt{net}_{o1}$ change with respect to $\mathtt{w}_5$? ($\frac{\partial \mathtt{net}_{o1}}{\partial \mathtt{w}_5}$)



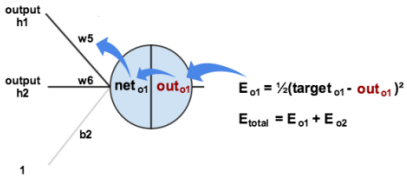
$$\frac{\partial \mathtt{E}_{\mathtt{total}}}{\partial \mathtt{w}_5} = \frac{\partial \mathtt{E}_{\mathtt{total}}}{\partial \mathtt{out}_{o1}} \times \frac{\partial \mathtt{out}_{o1}}{\partial \mathtt{net}_{o1}} \times \frac{\partial \mathtt{net}_{o1}}{\partial \mathtt{w}_5}$$

$$\mathtt{net}_{o1} = \mathtt{w}_5 \times \mathtt{out}_{h1} + \mathtt{w}_6 \times \mathtt{out}_{h2} + \mathtt{b}_2$$

$$\frac{\partial \mathtt{net}_{o1}}{\partial \mathtt{w}_5} = \mathtt{out}_{h1} = 0.59327$$

Backpropagation - Backward Pass - Output Layer (5/6)

- Putting it all together:

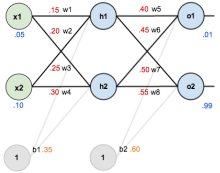


$$\frac{\partial E_{total}}{\partial w_5} = \frac{\partial E_{total}}{\partial out_{o1}} \times \frac{\partial out_{o1}}{\partial net_{o1}} \times \frac{\partial net_{o1}}{\partial w_5}$$

$$\frac{\partial E_{total}}{\partial w_5} = 0.74136 \times 0.18681 \times 0.59327 = 0.08216$$

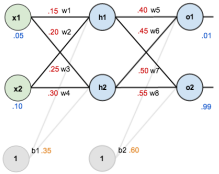
Backpropagation - Backward Pass - Output Layer (6/6)

- To decrease the error, we subtract this value from the current weight.



Backpropagation - Backward Pass - Output Layer (6/6)

- To decrease the error, we subtract this value from the current weight.
- We assume that the learning rate is $\eta = 0.5$.

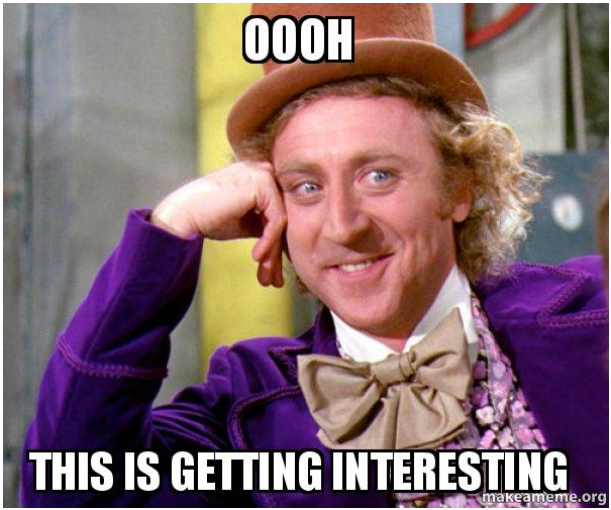


$$w_5^{(next)} = w_5 - \eta \times \frac{\partial E_{total}}{\partial w_5} = 0.4 - 0.5 \times 0.08216 = 0.35891$$

$$w_6^{(next)} = 0.40866$$

$$w_7^{(next)} = 0.5113$$

$$w_8^{(next)} = 0.56137$$

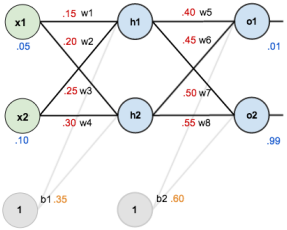


[https://makeameme.org/meme/oooh-this]

Backpropagation - Backward Pass - Hidden Layer (1/8)

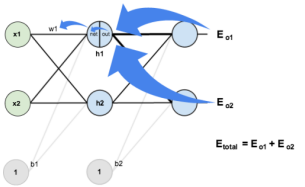
- Continue the backwards pass by calculating new values for w_1, w_2, w_3, and w_4.
- For w_1 we have:

$$\frac{\partial E_{total}}{\partial w_1} = \frac{\partial E_{total}}{\partial out_{h1}} \times \frac{\partial out_{h1}}{\partial net_{h1}} \times \frac{\partial net_{h1}}{\partial w_1}$$



Backpropagation - Backward Pass - Hidden Layer (2/8)

- Here, the output of each hidden layer neuron contributes to the output of multiple output neurons.
- E.g., out_{h1} affects both out_{o1} and out_{o2}, so $\frac{\partial E_{total}}{\partial \mathrm{out}_{h1}}$ needs to take into consideration its effect on the both output neurons.

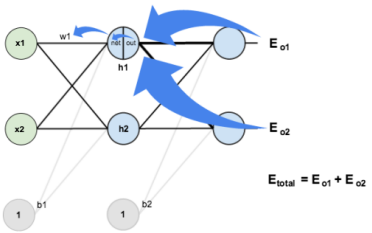


$$\frac{\partial E_{total}}{\partial w_1} = \frac{\partial E_{total}}{\partial \mathrm{out}_{h1}} \times \frac{\partial \mathrm{out}_{h1}}{\partial \mathrm{net}_{h1}} \times \frac{\partial \mathrm{net}_{h1}}{\partial w_1}$$

$$\frac{\partial E_{total}}{\partial \mathrm{out}_{h1}} = \frac{\partial E_{o1}}{\partial \mathrm{out}_{h1}} + \frac{\partial E_{o2}}{\partial \mathrm{out}_{h1}}$$

Backpropagation - Backward Pass - Hidden Layer (3/8)

- Starting with $\frac{\partial E_{o1}}{\partial out_{h1}}$



$$\frac{\partial E_{total}}{\partial out_{h1}} = \frac{\partial E_{o1}}{\partial out_{h1}} + \frac{\partial E_{o2}}{\partial out_{h1}}$$

$$\frac{\partial E_{o1}}{\partial out_{h1}} = \frac{\partial E_{o1}}{\partial out_{o1}} \times \frac{\partial out_{o1}}{\partial net_{o1}} \times \frac{\partial net_{o1}}{\partial out_{h1}}$$

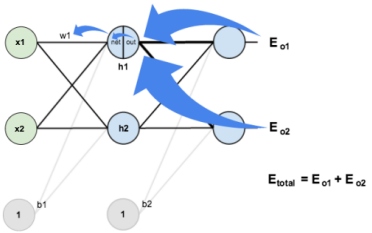
$$\frac{\partial E_{o1}}{\partial out_{o1}} = 0.74136, \frac{\partial out_{o1}}{\partial net_{o1}} = 0.18681$$

$$net_{o1} = w_5 \times out_{h1} + w_6 \times out_{h2} + b_2$$

$$\frac{\partial net_{o1}}{\partial out_{h1}} = w_5 = 0.40$$

Backpropagation - Backward Pass - Hidden Layer (4/8)

- Plugging them together.

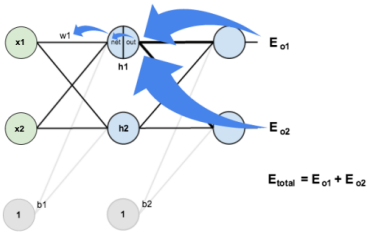


$$\frac{\partial \mathrm{E_{o1}}}{\partial \mathrm{out_{h1}}} = \frac{\partial \mathrm{E_{o1}}}{\partial \mathrm{out_{o1}}} \times \frac{\partial \mathrm{out_{o1}}}{\partial \mathrm{net_{o1}}} \times \frac{\partial \mathrm{net_{o1}}}{\partial \mathrm{out_{h1}}} = 0.74136 \times 0.18681 \times 0.40 = 0.0554$$

$$\frac{\partial \mathrm{E_{o2}}}{\partial \mathrm{out_{h1}}} = -0.01905$$

Backpropagation - Backward Pass - Hidden Layer (4/8)

- Plugging them together.



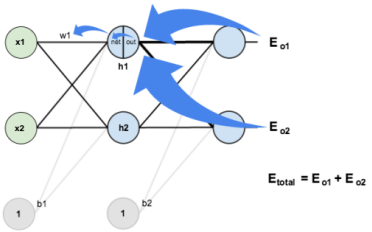
$$\frac{\partial E_{o1}}{\partial out_{h1}} = \frac{\partial E_{o1}}{\partial out_{o1}} \times \frac{\partial out_{o1}}{\partial net_{o1}} \times \frac{\partial net_{o1}}{\partial out_{h1}} = 0.74136 \times 0.18681 \times 0.40 = 0.0554$$

$$\frac{\partial E_{o2}}{\partial out_{h1}} = -0.01905$$

$$\frac{\partial E_{total}}{\partial out_{h1}} = \frac{\partial E_{o1}}{\partial out_{h1}} + \frac{\partial E_{o2}}{\partial out_{h1}} = 0.0554 + -0.01905 = 0.03635$$

Backpropagation - Backward Pass - Hidden Layer (5/8)

- Now we need to figure out $\frac{\partial \text{out}_{h1}}{\partial \text{net}_{h1}}$.



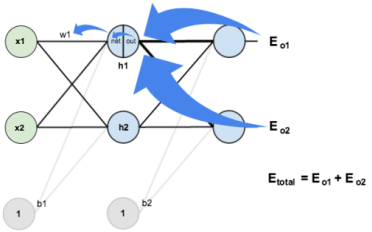
$$\frac{\partial E_{\text{total}}}{\partial w_1} = \frac{\partial E_{\text{total}}}{\partial \text{out}_{h1}} \times \frac{\partial \text{out}_{h1}}{\partial \text{net}_{h1}} \times \frac{\partial \text{net}_{h1}}{\partial w_1}$$

$$\text{out}_{h1} = \frac{1}{1 + e^{-\text{net}_{h1}}}$$

$$\frac{\partial \text{out}_{h1}}{\partial \text{net}_{h1}} = \text{out}_{h1}(1 - \text{out}_{h1}) = 0.59327(1 - 0.59327) = 0.2413$$

Backpropagation - Backward Pass - Hidden Layer (6/8)

- And then $\frac{\partial \text{net}_{h1}}{\partial w_1}$.



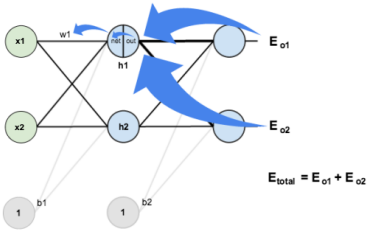
$$\frac{\partial E_{total}}{\partial w_1} = \frac{\partial E_{total}}{\partial out_{h1}} \times \frac{\partial out_{h1}}{\partial net_{h1}} \times \frac{\partial net_{h1}}{\partial w_1}$$

$$net_{h1} = w_1 x_1 + w_2 x_2 + b_1$$

$$\frac{\partial net_{h1}}{\partial w_1} = x_1 = 0.05$$

Backpropagation - Backward Pass - Hidden Layer (7/8)

- Putting it all together.

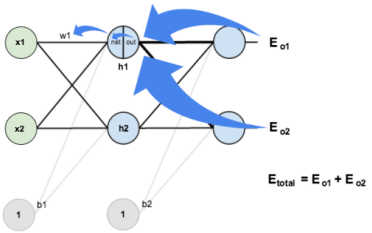


$$\frac{\partial \mathrm{E_{total}}}{\partial \mathrm{w_1}} = \frac{\partial \mathrm{E_{total}}}{\partial \mathrm{out_{h1}}} \times \frac{\partial \mathrm{out_{h1}}}{\partial \mathrm{net_{h1}}} \times \frac{\partial \mathrm{net_{h1}}}{\partial \mathrm{w_1}}$$

$$\frac{\partial \mathrm{E_{total}}}{\partial \mathrm{w_1}} = 0.03635 \times 0.2413 \times 0.05 = 0.00043$$

Backpropagation - Backward Pass - Hidden Layer (8/8)

- We can now update w_1.
- Repeating this for w_2, w_3, and w_4.



$$w_1^{(\text{next})} = w_1 - \eta \times \frac{\partial E_{\text{total}}}{\partial w_1} = 0.15 - 0.5 \times 0.00043 = 0.14978$$

$$w_2^{(\text{next})} = 0.19956$$

$$w_3^{(\text{next})} = 0.24975$$

$$w_4^{(\text{next})} = 0.2995$$



Summary



Summary

- LTU
- Perceptron
- Perceptron weakness
- MLP and feedforward neural network
- Gradient-based learning
- Backpropagation: forward pass and backward pass
- Output unit: linear, sigmoid, softmax
- Hidden units: sigmoid, tanh, relu



Reference

- Hands-on machine learning with Scikit-Learn and TensorFlow (Ch. 10)



Questions?